Rutherford Scattering Simulation

Group project of: **Zsolt** Maraczi-**Ábel** Palatinszky-**István** Polt

- **Goal of the program:**

Simulate and visualize Ernest Rutherford's experiment from 1911: The program models alpha-particle trajectories in a 2D plane as they are scattered by Coulomb forces from nuclei positioned at given coordinates.

- **Philosophy of the division of tasks**

In our task we have chosen to divide the workflow into four main phases. We wanted everyone to participate in every part of the project to deepen his understanding in each topic, so we decided that everyone will take part in every phase, with each phase divided into three separate tasks. We hope that this approach will ensure an equal distribution of workload among the participants and a common understanding of every part of the project.

- **The Program**

1.1 The Physics behind the program

1.11 The first phase focuses on implementing the physical simulation. This part will serve as the foundation of our project. We will implement mathematical and physical solutions to simulate the movement of alpha particles under the influence.

- Planned libraries <math.h, stdlib.h, float.h>

1.12 **Zsolt:** Calculation of the Coulomb forces exerted by the nuclei and the resulting acceleration of alpha particles.

- Calculate the net force F(r) on the particle by all nuclei. From $F(r) = \Sigma i$ $Fi(r)$ and $Fi(r) = k * (q\alpha * Zi * e) * (r - Ri) / |r - Ri|3$ where r = (x, y): position of the alpha particle with velocity v = (vx, vy), Zi: atomic number of nuclei i at position Ri = (Xi, Yi) with charge +Zi e. Also, e is the elementary charge, k = / (4 π ε0) if using SI; qα = +2 e.

1.13 **Ábel:** Implementation of the numerical integration method (RK4) to update particle positions and velocities.

- From the force calculate the speed and then the new position from there, with the Runge-Kutta 4 method based on Newton's Equation of motion: dr/dt = v, dv/dt = F(r)/m.

1.14 **István:** Initialization of alpha particles, management of the simulation loop, and determination of termination conditions.

- Repeat updating and storing the data for the alpha particles until the max steps, using dt (both given by the user).

1.2 User Input and File Handling

1.21 Once the simulation is functional, we will implement user interaction and data handling. We will write the required functions to read information from external files and convert it into data structures that we can better access during the program. We will also try to validate inputs so the program can handle incorrect inputs without crashing.

- Planned libraries: < stdio.h, stdlib.h, string.h, errno.h, limits.h>

1.22 **Zsolt**: Implementation of user input for simulation parameters, including dimensions, particle count, initial velocity, time step, and maximum number of steps.

1.23 **Ábel**: Reading and processing the nuclei data file, including nucleus coordinates and atomic numbers.

1.24 **István**: Input validation and handling invalid parameters, missing files, and other potential errors.

All Inputs:

When receiving inputs if the value given by the user doesn't match the requirements given in the () next to them, the program should terminate with specific error massage.

Also note that certain value limitations will have to be tested, to make sure the simulation doesn't take too long. These will be marked with *.

- From the Nuclei data input file:
  - One nucleus per line format:
    x y Z
    - **x**, **y** – coordinates of the nucleus centre (simulation units)
    - **Z** – atomic number (positive charge, defaults to 79 if missing)

- From user:
  - Path for Nuclei data input file (file path must be valid)
  - Path for PPM output file (file path must be valid)
  - Image width and height - in pixels (integer, >0, <too large*)
  - Physical width and height - in simulation units (integer, >0, <too large*)
  - Number of alpha particles to simulate (integer,>0, <too many*)
  - Start position of alpha particles (integer, between the given simulation sizes)
  - Initial velocity magnitude (double, >0, <too large*)
  - Time step **dt** (double, >0, <1)
  - Maximum number of integration steps (integer, >0, <too large* - based on dt)

1.3 Visualization and Output

1.31 The third phase focuses on generating a graphical representation of the simulation results. The visualization of the results and its supporting functions with the ability to export these results will be written. By the end of this phase the program will be able to produce a complete

- Planned libraries < stdio.h, stdlib.h, math.h, limits.h>

1.32 **Zsolt**: Conversion of physical coordinates into image pixel coordinates.

1.33 **Ábel**: Visualization of particle trajectories using different colours and representation of atomic nuclei.

1.34 **István**: Generation and saving of the final image in PPM format.

All outputs:

- PPM file:
  - Black background
  - White dots for nucleus positions
  - Curved coloured lines representing alpha particle trajectories
  - Each alpha particle with a unique or smoothly varying colour

1.4 Integration, Testing and Debugging

1.41 In the final phase, we will combine the individual components into a complete program and test its functionality. We will test the program verifiyin its accuracy. In the general debugging everybody will participate. By the end of this phase, our goal is to have a stable and reliable program that meets all the requirements of the assignment.

- Planned libraries: < assert.h, math.h, stdio.h>

1.42 **Zsolt**: Verification of the physical calculations and simulation results.

1.43 **Ábel**: Testing user inputs, file handling, and error management.

1.44 **István**: Testing image generation and checking the integration of the program components.

1.45 Throughout the project, we will use GitHub. Everybody will work on a separate branch, allowing us to work independently. Completed components will be merged into the main branch after review and testing. We hope that this approach will help us keep track of changes.